HOMEWORK-2 REPORT

ABOUT THE PROJECT

Purpose

The purpose of this assignment is to implement a double linked list data structure from scratch and perform various operations on this structure. The goal is to develop a LinkedList class manually without using Java's built-in libraries.

Development Environment

- **IDE:** NetBeans
- **Programming Language:** Java
- **JDK Version:** Java SE 8 or higher

SOLUTION APPROACH

Data Structure Design

The basic structure used in the project is the **DLNode** (Double Linked Node) class:

```
public class DLNode {
   public int Element;      // Value of the node
   public DLNode left;      // Reference to previous node
   public DLNode right;     // Reference to next node
}
```

The **LinkedList** class contains the following variables:

- head: Points to the beginning of the list
- tail: Points to the end of the list
- size: Stores the number of elements in the list

DETAILED METHOD DESCRIPTIONS

Insert(int newElement, int pos)

Purpose: Inserts a new element at the specified position.

Algorithm:

1. Position validation is performed ($0 \leq pos \leq size$)
2. A new node is created
3. Special cases are checked:
    - If list is empty → head and tail point to the new node
    - If pos = 0 → Insert at the beginning
    - If pos = size → Insert at the end
    - Other cases → Insert in the middle

4. Pointers are updated

Time Complexity: O(n) - In the worst case, traverses to the end of the list

Delete(int pos)

Purpose: Deletes the element at the specified position and returns its value.

Algorithm:

1. Position validation ($0 \leq pos < size$)
2. The node to be deleted is found
3. Special cases:
    - If only one element exists → Empty the list
    - If pos = 0 → Delete from the beginning
    - If pos = size-1 → Delete from the end
    - Other cases → Delete from the middle
4. Pointers are rearranged
5. The deleted value is returned

Time Complexity: O(n)

ReverseLink()

Purpose: Reverses the order of the list.

Algorithm:

1. Swaps left and right pointers for each node
2. Traverses the list from beginning to end
3. For each node:
    - Swaps left and right using a temporary variable
    - Moves to the next node (now using left pointer)
4. Finally, head and tail are swapped

Example:

- Before: [2 3 2 2 1] → head:2, tail:1
- After: [1 2 2 3 2] → head:1, tail:2

Time Complexity: O(n)

SquashL()

Purpose: Compresses consecutive identical elements and creates (element, count) pairs.

Algorithm:

1. A new list is initialized
2. The original list is traversed from beginning to end

3. For each element:
    - Counts how many times it repeats consecutively
    - Adds to the new list as (element, count)
4. The old list is replaced with the new list

Example:

- Input: [1 2 3 3 3 3 2 2 2 2 1 2 2 3 2]
- Output: [1 1 2 1 3 4 2 4 1 1 2 2 3 1 2 1]
    - 1 occurrence of 1, 1 occurrence of 2, 4 occurrences of 3, 4 occurrences of 2, etc.

Time Complexity: O(n)

OplashL()

Purpose: Performs the reverse of SquashL. Expands (element, count) pairs.

Algorithm:

1. A new list is initialized
2. The current list is traversed in pairs (element and count)
3. For each pair:
    - Adds the element to the new list count times
4. The old list is replaced with the new list

Example:

- Input: [1 1 2 1 3 4 2 4 1 1 2 2 3 1 2 1]
- Output: [1 2 3 3 3 3 2 2 2 2 1 2 2 3 2]

Time Complexity: O(n*k) - k: average repetition count

Output() and ROutput()

Output(): Prints the list from beginning to end **ROutput():** Prints the list from end to beginning (using left pointer from tail)

Both methods do not modify the list structure, they only provide console output.

Time Complexity: O(n)

toString()

Returns the list elements as a String. Similar logic to Output() but writes to String instead of console.

LinkedListException()

Creates a custom exception to be thrown in error situations:

return new HW2Exception("Not supported yet.");

SPECIAL CASES AND ERROR HANDLING

Invalid Position Check

- For Insert: pos < 0 || pos > size
- For Delete: pos < 0 || pos >= size

Empty List Checks

- All methods check for empty list condition
- Methods like Delete and ROutput throw exceptions on empty list

Pointer Updates

- Both left and right pointers are correctly updated in every insert/delete operation
- Head and tail are specially checked for edge cases

TEST RESULTS

Expected output with the provided test program:

```
hw2.HW2Exception: Not supported yet.
The Elements in the list are : 2 3 2 2 1 2 2 2 2 3 9 9 3 3 3 2 1
The Elements in the list are : 1 2 3 3 3 9 9 3 2 2 2 2 1 2 2 3 2
The Elements in the list are : 1 1 2 1 3 3 9 2 3 1 2 4 1 1 2 2 3 1 2 1
The Elements in the list are : 1 2 3 3 3 9 9 3 2 2 2 2 1 2 2 3 2
The Reverse Elements in the list are : 2 3 2 2 1 2 2 2 2 3 9 9 3 3 3 2 1
1 2 3 3 3 9 9 3 2 2 2 2 1 2 2 3 2
```

COMPLEXITY ANALYSIS

Method	Time Complexity	Space Complexity
Insert	O(n)	O(1)
Delete	O(n)	O(1)
ReverseLink	O(n)	O(1)
SquashL	O(n)	O(n)
OplashL	O(n*k)	O(n)
Output	O(n)	O(1)
ROutput	O(n)	O(1)

CONCLUSION

In this project, the double linked list data structure has been successfully implemented. All methods work as expected and the test program produces correct outputs.

Topics Learned:

- Double linked list structure
- Pointer management
- Exception handling
- Algorithm design and optimization

Important Considerations:

- Java's built-in libraries were not used
- Stack and Queue data structures were not used
- All implementation was written from scratch

Prepared by: Harun Yavaş
Student ID: 36445399294
Date: 25/11/2025